

Tute (Week 8)

Software and Systems Modelling

This tutorial is themed around RFC 677, in which Paul Johnson and Robert Thomas outline a method for maintaining consistent replicas of a distributed database. To the best of my knowledge, it is the first published solution to this problem, which has only grown in importance as distributed systems have taken over the world. These days though, there are more modern algorithms to do the job.

Here is my attempt to summarise the salient points:

1. The system consists of n independent *database management processes* (DBMPs) with unique PIDs. They communicate with each other using asynchronous message passing over reliable channels with in-order message delivery.
2. Each DBMP maintains a local copy of the database, as a mapping from keys $k \in \mathbf{K}$ to pairs $(v, (t, p))$ of *values* $v \in \mathbf{V}$ and *timestamps* $(t, p) \in (\mathbb{N} \times \mathbb{N})$. In the initial state, all the DBMPs have identical local copies.
3. Each DBMP with PID i maintains a logical clock $C_i \in \mathbb{N}$ that is incremented with every action performed by the DBMP. Clocks are not synchronized between different DBMP:s.
4. At any point, a DBMP with PID i may initiate a *modification request* to update the database, requesting a new value v to be assigned to key k . When it does so, it sends a message $(k, v, (C_i, i))$ to every other DBMP.
5. When a process receives (or initiates) a modification request for key k , it compares the timestamp of the request with the timestamp of the pre-existing entry for k in its local database. If the timestamp of the modification is lexicographically larger, the database entry is updated with the new (value, timestamp); otherwise, the modification is discarded.

A PID is a process ID, a natural number that uniquely identifies a DBMP.

A pair (n, m) is lexicographically larger than (n', m') if either $n > n'$, or if $n = n'$ and $m > m'$. This is similar to how words are sorted in a dictionary.

Question 1

Sketch a formalisation of points 1-5 as a labelled transition system. (Don't do the whole thing — you'll get bogged down in technical details.)

There's many ways to do this. Here's one.

Let K and V denote the sets of keys and values, respectively.

The state is a set of local states for each DBMP, where each DBMP has

- A $PID \in \mathbb{N}$
- A local clock value $\in \mathbb{N}$
- A database db , which is a function $K \rightarrow V \times (\mathbb{N} \times \mathbb{N})$
- A list of incoming messages $\in \text{List}(K \times V \times (\mathbb{N} \times \mathbb{N}))$. (Let's not worry about how to formalise lists in set theory here).

The incoming messages aren't physically part of the DBMP, of course — they're out there in the network somewhere. Nonetheless, it's convenient to model it as a queue.

Then, in our labelled transition system $(S, \rightarrow)$ we have

$$S = \square(\mathbb{N} \times \mathbb{N} \times (K \rightarrow V \times (\mathbb{N} \times \mathbb{N})) \times \text{List}(K \times V \times (\mathbb{N} \times \mathbb{N})))\$$$

This model comes with some caveats. For example, we don't want S to contain more than one element with the same PID, but we haven't forbidden it. To achieve this, we could phrase S as a partial function from PIDs instead.

We'll define the transition relation using the following inference rules.

The first inference rule represents a DBMP receiving, and accepting, a modification request:

$$\frac{db(k) = (v', (ck'', pid'')) \quad (ck'', pid'') < (ck', pid') \quad \forall p \in \text{procs}. pid \neq p_1}{(pid, ck, db, (k, v, (ck', pid'))) :: \text{msgs} \cup \text{procs} \rightarrow (pid, ck + 1, db[k \mapsto (v, (ck', pid'))], \text{msgs} \cup \text{procs}}$$

The next inference rule represents a process rejecting a modification request:

$$\frac{db(k) = (v', (ck'', pid'')) \quad (ck'', pid'') > (ck', pid') \quad \forall p \in \text{procs}. pid \neq p_1}{(pid, ck, db, (k, v, (ck', pid'))) :: \text{msgs} \cup \text{procs} \rightarrow (pid, ck + 1, db, \text{msgs}) \cup \text{procs}}$$

The final inference rule represents a process creating a modification request:

$$\frac{}{(pid, ck, db, \text{msgs}) \cup \text{procs} \rightarrow (pid, ck + 1, db, \text{msgs} + [(k, v, (ck, pid))]) \cup (pid, ck, db, \text{msgs} + [(k, v, (ck, pid))]) : (pid, ck, db, \text{msgs}) \in \text{procs}}$$

Note that this rule doesn't actually perform the local update, instead it sends the modification request to itself as well as to all the other processes. This is a deliberate simplification, to avoid having two inference rules doing essentially the same work.

There is one more aspect not captured by these inference rules: the rules for accepting and rejecting requests always take the first (oldest) element from the message list. However, it's possible for a message other than the oldest to arrive first, if its DBMP of origin is different. To model this, we would have to allow messages to be consumed from not just the first element in the list, under certain circumstances.

In the above, + denotes list append, and :: denotes list cons (inserting an element to the front of a list).

Question 2

If you read the RFC, you'll find several details that the summary above completely ignores, and at least one point where the summary chooses a particular interpretation of a piece of text. See how many you can spot.

For one thing, the summary above only considers modification requests originating from what the RFC calls assignments. Selection, creation and deletion requests are completely ignored.

As for ambiguities, consider the following quote:

"When the assignment is initiated (by a person or process) the DBMP involved makes the change locally, and creates a copy of the modified entry and an associated list of DBMPs to which the change must be sent."

When that says "make the change locally", does it mean make the change no matter what? Or does it mean make the change, unless the entry we already have has a higher timestamp?

We chose the latter for a number of reasons.

1. The algorithm doesn't satisfy eventual consistency if timestamps are ignored for local updates: a DBMP may then make a local update that every other process rejects.
2. The following quote comes later, and seems to suggest interpretation 2:

"When a DBMP receives an assignment modification (either locally or from another DBMP) it compares the timestamp of the modification with the timestamp of the copy of the entry in its database and keeps whichever is more "recent" as defined by the ordering given above"

That quote suggests, but doesn't make 100% clear, that interpretation #2 is intended depending on what exactly is meant by a local modification.

Another point: a literal reading of the text seems to suggest modification requests are always sent out when a local update is attempted, even one that will clearly be rejected because the DBMP of origin already had a more recent entry. That's what we went with. But there are at least two other readings possible:

1. Modification requests are only sent to others if it would have a higher timestamp than the existing entry in our local database, i.e., if our local update would succeed.
2. If a local modification fails, an update is still sent to others— but with the existing entry in our database, not with our proposed modification.

Question 3

This algorithm guarantees that if two modification requests have the same timestamp, the modification requests are equal. How?

If two modification requests $(k, v, (n, pid))$ and $(k', v', (n, pid))$ have equal timestamps, then (because PIDs are unique) they must originate from the same DBMP. Because clocks are incremented with every action performed, the two requests must also originate with the same action. The only action that sends messages is clause 4 above, which sends identical messages.

Question 4

The algorithm is eventually consistent, in the following sense: if modifications cease and all outstanding messages have been received and acted upon, all DBMPs will have identical local copies of the database.

Define an invariant that can be used to prove eventual consistency. The invariant must:

- be true in the initial state,
- be preserved by the acts of sending and receiving modification requests, and
- imply eventual consistency once there are no more in-flight messages.

Prove these points.

The invariant can be stated something like this. For every key k there is a unique v associated with the lexicographically highest timestamp (v, ts) of a message or database entry involving k . For every DBMP i that does *not* have (v, ts) associated with k , there is an in-flight message to i containing the modification request (k, v, ts) .

If there are no in-flight messages, this invariant implies that every DBMP must have identical databases. Proof by contradiction: if not, there must a DBMP that has an entry with a lower timestamp than the maximum for some key k . This DBMP must have an inbound modification request, which contradicts the invariant.

It remains to show that the invariant is preserved by actions 4 and 5, which is relatively straightforward.

Question 5

While the algorithm as presented above is *eventually* consistent it is not *causally* consistent. This means that there are executions where a modification request m_1 is discarded in favour of another modification request m_2 , even if the DBMP where m_1 originated was aware of the request m_2 before m_1 fired.

Give an example execution where a causally inconsistent behaviour occurs.

Suppose there are three DBMPs with PIDs 0, 1 and 2. Initially they all map the key k to the value `"hi"`. Then consider the following interleaving:

1. DBMP 0 sends request $(k, \text{"hi"}, (5, 0))$, and increments its clock value from 5 to 6.
2. DBMP 1 and 2 receive this message from 0, and update their local databases accordingly.
3. DBMP 1 sends request $(k, \text{"hej"}, (3, 1))$, and increments its clock value from 3 to 4.
4. DBMP 2 receives $(k, \text{"hej"}, (3, 1))$, and discards the update because $3 < 5$.

This is causally inconsistent because, even though the out-of-synch clocks say otherwise, the modification request from DBMP 1 demonstrably happens after that of DBMP 0. Since sending of the former precedes receipt of the latter, there may even be a direct causal link. It could be that DBMP 1 decided to send its request only because of the request it received from DBMP 0.

If you're interested in how to think about causality in distributed systems, and how to set up the clocks so that they respect causality, the canonical reference is Leslie Lamport. But that's beyond the scope of the course.